LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Smart Parking Space Rental & License Validator

Student: Sanjith Suresh Kumar | Roll No.: 22

1. Selected Architectural Style: Microservices Architecture

Microservices Architecture is selected for the Smart Parking Space Rental & License Validator because the system contains several distinct responsibilities that can operate as independently managed services. Parking rental, parking-space availability, license validation, payment processing, and warden operations can therefore be separated into focused components with clear APIs. This supports independent deployment and scaling while keeping the system responsive during periods of high parking demand.

2. Component Model

The component model contains the following major services: Parking Rental Manager, License Validator, Parking Availability Manager, Payment Service, and Warden Management Service. Supporting components include the Parking Space Database, License Registry Database, and Payment Gateway. The main actors are the Vehicle Owner and Parking Warden. Key interfaces include LicenseValidationAPI, SpaceAvailabilityAPI, PaymentAPI, and SpaceManagementAPI.

3. Justification for the Choice

- Separation of responsibilities: Each major business function is isolated into a dedicated service, making the architecture easier to develop, test, maintain, and deploy.
- Independent scalability: Parking availability and rental services may experience substantially higher traffic during peak hours. They can be scaled independently without scaling the entire application.
- Fault isolation: A temporary failure in payment or license validation does not require the complete parking platform to become unavailable. Services can handle failures and recover independently.
- Integration flexibility: Well-defined APIs allow the system to communicate with external license registries and payment gateways without tightly coupling those systems to the core parking application.

4. Security Implications

The architecture must protect vehicle-owner information, license data, rental records, and payment-related transactions. Service-to-service communication should use authenticated and encrypted APIs. Role-based access control should restrict warden functions to authorized personnel, while license and payment services should expose only the minimum information required by other services. Payment credentials should not be stored by the parking application; the Payment Service should communicate securely with the external payment gateway. Audit logs should record important rental, validation, and administrative actions.

5. Performance Implications

Microservices allow performance-critical services to be optimized separately. The Parking Availability Manager can use caching or fast data access for frequently requested space-status information, while the Rental Manager can scale independently when many users attempt to reserve spaces simultaneously. Separating license validation and payment processing also prevents these comparatively slower external operations from blocking unrelated availability queries. API timeouts, retries, and asynchronous notifications can further improve resilience under load.

6. Conclusion

Microservices Architecture is appropriate for the Smart Parking Space Rental & License Validator because it provides clear separation of system responsibilities, independent scalability, controlled access to sensitive information, and better fault isolation. The selected component structure therefore supports the functional and non-functional requirements of a smart-city parking platform.

